COMPUTER ENGINEERING

QUEZON CITY

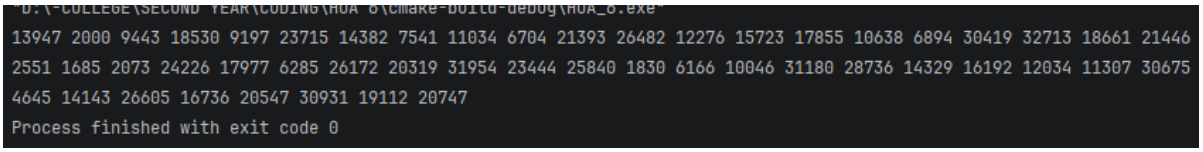
COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	8/18/2026	**DATE SUBMITTED:**	8/26/2026
NAME:	Al Twain D. Pasion	**INSTRUCTOR:**	Engr Jimlord Quejado

ACTIVITY NO. 6 Searching Algorithm

1. PROCEDURE OUTPUT

What procedures did you perform?

Table 6-1. Data Generated and Observations.

SCREENSHOT


```
"D:\~COLLEGE\SECOND YEAR\CODING\HOA 6\cmake-build-debug\HOA_6.exe"
13947 2000 9443 18530 9197 23715 14382 7541 11034 6704 21393 26482 12276 15723 17855 10638 6894 30419 32713 18661 21446
2551 1685 2073 24226 17977 6285 26172 20319 31954 23444 25840 1830 6166 10046 31180 28736 14329 16192 12034 11307 30675
4645 14143 26605 16736 20547 30931 19112 20747
Process finished with exit code 0
```

OBSERVATION
I've tried running it a few more times before the screenshot. I observed that it can generate numbers that have 1-digit to 5-digits.

Table 6-2a. Linear Search for Arrays

CODE

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "nodes.h"
#include "searchingAlgo.h"

const int max_size = 50;

int main() {

    int dataset[max_size];
    srand( Seed: time(0));
    for (int i = 0; i < max_size; i++) {
        dataset[i] = rand();
    }
    for (int i = 0; i < max_size; i++) {
        std::cout << dataset[i] << " ";
    }

    int toFind;
    std::cout << "\nValue to search: ";
    std::cin >> toFind;

    linearSearchArr(dataset, max_size,  datafind: toFind);

    return 0;
}
```

```cpp
template <typename T>
void linearSearchArr(T data[], int arrSize, T datafind) {
    int i = 0;
    while (i < arrSize) {
        if (data[i] == datafind) {
            std::cout << "Search Success\n";
            return;
        }
        i++;
    }
    std::cout << "Element not found\n";
}
```

OUTPUT

```
D:\~COLLEGE\SECOND YEAR\CODING\NOA 6\cmake-build-debug\NOA_6.exe
17575 15928 31898 14184 32657 28168 31675 32139 5330 24713 20909 9882 32567 15843 31811 5673 15572 20843 2259 5705 22856
 31441 4246 29274 27971 16282 21585 13345 14149 4305 15157 14107 12712 6394 14596 1552 18912 9800 24438 14058 7426 1296
7899 14536 23939 24188 31233 18511 17986 28548
Value to search:1296

Search Success

Process finished with exit code 0
```

OBSERVATION

The random number generator generated 50 elements for us to use our linear search for array function. The output first displays all the elements in our array and then asks for a value we need to find. We pick a random number from the array to see if it would find it, after entering the value the search was a success.

Table 6-2b. Linear Search for Linked List

CODE

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "nodes.h"
#include "searchingAlgo.h"

const int max_size = 50;

int main() {
    /*
    int dataset[max_size];
    srand(time(0));
    for (int i = 0; i < max_size; i++) {
        dataset[i] = rand();
    }
    for (int i = 0; i < max_size; i++) {
        std::cout << dataset[i] << " ";
    }
    */
    Node<char>* name1 = new_Node( newData: 'T');
    Node<char>* name2 = new_Node( newData: 'W');
    Node<char>* name3 = new_Node( newData: 'A');
    Node<char>* name4 = new_Node( newData: 'I');
    Node<char>* name5 = new_Node( newData: 'N');

    name1->next = name2;
    name2->next = name3;
    name3->next = name4;
    name4->next = name5;

    std::cout << "Linear Search for LL result:\n";
    linearSearchLL(name1,  datafind: 'I');
    return 0;
}
```

```cpp
template <typename T>
void linearSearchLL(Node<T>* node, T datafind) {
    while (node != nullptr) {
        if (node->data == datafind) {
            std::cout << "Search Success\n";
            return;
        }
        node = node->next;
    }
    std::cout << "Element not found\n";
}
```

OUTPUT

```
"D:\-COLLEGE\SECOND YEAR\CODING\HOA 6\cmake-build-debug\HOA_6.exe"
Linear Search for LL result:
Search Success

Process finished with exit code 0
```

OBSERVATION

The logic behind linear search in linked lists is essentially the same as the linear search array. It traverses through all the nodes and sees if the data to find is the same as the data of the current node, if not it will move on to the next node. For this scenario, we are to find the letter “l”, now the function will traverse to each node until it either finds the data or has reached the end that will stop the while function and print that the element has not been found in the linked list.

\

Table 6-2b. Binary Search for Arrays

CODE

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "nodes.h"
#include "searchingAlgo.h"

const int max_size = 50;

int main() {

    int dataset[max_size];
    srand( Seed: time(0));
    for (int i = 0; i < max_size; i++) {
        dataset[i] = rand();
    }
    for (int i = 0; i < max_size; i++) {
        std::cout << dataset[i] << " ";
    }

    int toFind;
    std::cout << "\nValue to search: ";
    std::cin >> toFind;

    bubbleSort(dataset, max_size);
    binarySearchArr(dataset, max_size, toFind);

    return 0;
}
```

```cpp
template <typename T>
void binarySearchArr(T data[], int arrSize,T dataFind) {
    int low = 0;
    int high = arrSize - 1;

    while (low <= high) {
        int middle = getMiddleArr(low, high);

        if (data[middle] == dataFind) {
            std::cout << "Search Success\n";
            return;
        }
        else if (data[middle] > dataFind) {
            high = middle - 1;
        }
        else {
            low = middle + 1;
        }
    }

    std::cout << "Search Failed\n";
}
```

```cpp
template <typename T>
void bubbleSortArr(T data[], int arrSize) {
    for (int i = 0; i < arrSize - 1; i++) {
        for (int j = 0; j < arrSize - 1 - i; j++) {
            if (data[j] > data[j + 1]) {
                std::swap( a: &data[j], b: &data[j + 1]);
            }
        }
    }
}
```

OUTPUT

```
"D:\-COLLEGE\SECOND YEAR\CODING\HOA 6\cmake-build-debug\HOA_6.exe"
27473 22956 12697 8520 2692 2534 28330 6929 15225 2144 31357 12315 26636 32717 23551 30323 3501 31024 13377 5813 20244 3
0388 32203 24086 20017 13783 22066 12712 25186 23758 1517 18138 32489 5925 14948 17726 23965 26023 9802 13712 16863 9597
 31789 25075 26576 4762 30539 2941 29634 18774
Value to search:32203

Search Success

Process finished with exit code 0
```

OBSERVATION

Since this is a “Binary Search”, one of the requirements I need to fulfill is that the elements should be ordered from lowest to highest. I solved this by making a simple bubble sort. The actual function itself successfully does what a binary search would do. It’ll get the highest and lowest value of the elements and get their middle. After that, it replaces the high or low value with the middle depending if it is greater or lesser than the value to find. It will do this until the middle is equal to the value to find, or if the low and high values are the same or have nothing in between them meaning that the value to find does not exist in the given elements..

Table 6-2b. Binary Search for Linked List

CODE

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "nodes.h"
#include "searchingAlgo.h"

const int max_size = 50;

int main() {

    Node<char>* name1 = new_Node( newData: 'A');
    Node<char>* name2 = new_Node( newData: 'B');
    Node<char>* name3 = new_Node( newData: 'E');
    Node<char>* name4 = new_Node( newData: 'H');
    Node<char>* name5 = new_Node( newData: 'X');

    name1->next = name2;
    name2->next = name3;
    name3->next = name4;
    name4->next = name5;

    binarySearchLL(name1,  dataFind: 'E');

    return 0;
}
```

```cpp
template <typename T>
void binarySearchLL(Node<T>* head, T dataFind) {
    Node<T>* start = head;
    Node<T>* last = nullptr;

    while (start != last) { //if start == last, it means that they did not find the data
        Node<T>* middle = getMiddleLL(start, last);

        if (middle->data == dataFind) {
            std::cout << "Search Success\n";
            return;
        }
        else if (middle->data > dataFind) {
            last = middle;
        }
        else { //middle->data < dataFind
            start = middle->next;
        }
    }
    std::cout << "Search Failed\n";
}
```

OUTPUT

```
"D:\-COLLEGE\SECOND YEAR\CODING\HOA 6\cmake-build-debug\HOA_6.exe"
Search Success

Process finished with exit code 0
```

OBSERVATION

So after a bit of digging, the binary search for linked lists came to be like this. It works even with character data types because it uses the ASCII code of each letter. So when you compare the data to another, it can determine which one is greater or less than the other. This only works if the elements are all uppercase OR lowercase, never both. Similar to the binary search for arrays, this also needs to be arranged in a way. So I entered alphabetically ordered data.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

For each provided problem, give a screenshot of your code, the output console, and your answers to the questions.

Problem 1. Suppose you are doing a sequential search of the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]. Utilizing both a linked list and an array approach to the list, use sequential search and identify how many comparisons would be necessary to find the key '18'?

CODE

```cpp
#include <iostream>
#include "nodes.h"
#include "searchingAlgo.h"

const int size = 10;

int main() {

    //given data set
    int dataset[size] = {15,18,2,19,18,0,8,14,19,14};

    Node<int>* head = new_Node(dataset[0]);
    Node<int>* tail = head;

    for (int i = 1; i < size; i++) {
        tail->next = new_Node(dataset[i]);
        tail = tail->next;
    }

    std::cout << "Linear Search for '18' in Array:\n";
    linearSearchArr(dataset, size, datafind: 18);

    std::cout << "Linear Search for '18' in Linked List:\n";
    linearSearchLL(head, datafind: 18);


    return 0;
}
```

```cpp
template <typename T>
void linearSearchArr(T data[], int arrSize, T datafind) {
    int i = 0;
    while (i < arrSize) {
        if (data[i] == datafind) {
            std::cout << "Search Success\n";
            std::cout << "Comparison Count: " << i+1 << "\n";
            return;
        }
        i++;
    }
    std::cout << "Element not found\n";
    std::cout << "Comparison Count: " << i << "\n";
}
```

```cpp
template <typename T>
void linearSearchLL(Node<T>* node, T datafind) {
    int counter = 1;
    while (node != nullptr) {
        if (node->data == datafind) {
            std::cout << "Search Success\n";
            std::cout << "Comparison Count: " << counter << "\n";
            return;
        }
        node = node->next;
        counter++;
    }
    std::cout << "Element not found\n";
    std::cout << "Comparison Count: " << counter-1 << "\n";
}
```

OUTPUT

```
Linear Search for '18' in Array:
Search Success
Comparison Count: 2
Linear Search for '18' in Linked List:
Search Success
Comparison Count: 2
```

ANSWER:
It would take 2 comparisons for both array and linked list to find the key “18” in a linear search.

Problem 2. Modify your sequential search algorithm so that it returns the count of repeating instances for a given search element ‘k’. Test on the same list given in problem 1.

CODE

```
std::cout << "Modified Linear Search for counting '18's in Array:\n";
linearSearchArrMOD(dataset, size, datafind: 18);

std::cout << "Modified Linear Search for counting '18's in Linked List:\n";
linearSearchLLMOD(head, datafind: 18);

return 0;
```

```
template <typename T>
void linearSearchArrMOD(T data[], int arrSize, T datafind) {
    int i = 0;
    int counter = 0;
    while (i < arrSize) {
        if (data[i] == datafind) {
            counter++;
        }
        i++;
    }
    std::cout << "Count: " << counter << "\n";
}
```

```
template <typename T>
void linearSearchLLMOD(Node<T>* node, T datafind) {
    int counter = 0;
    while (node != nullptr) {
        if (node->data == datafind) {
            counter++;
        }
        node = node->next;
    }
    std::cout << "Count: " << counter << "\n";
}
```

OUTPUT

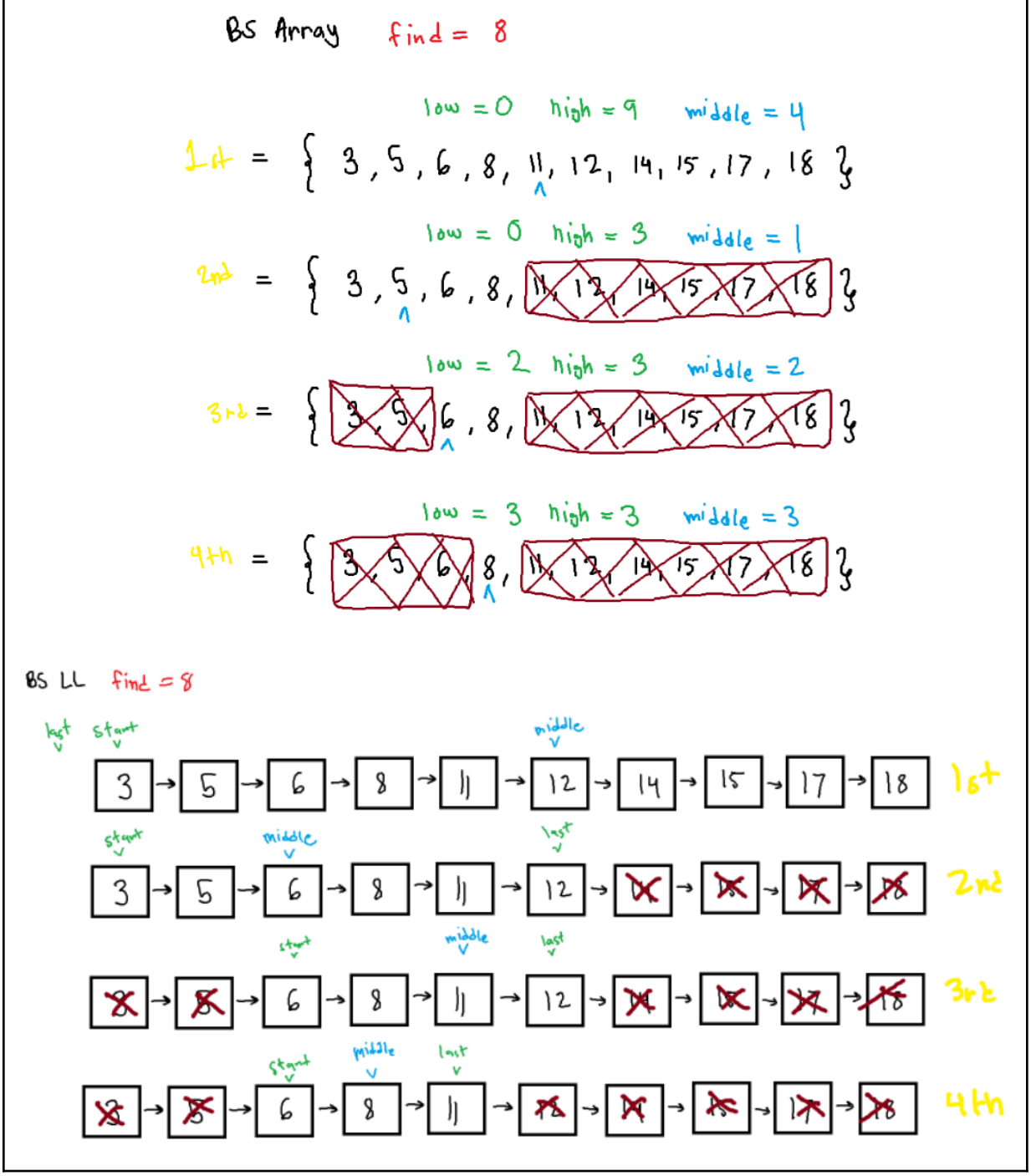
```
Linear Search for '18' in Array:
Search Success
Comparison Count: 2
Linear Search for '18' in Linked List:
Search Success
Comparison Count: 2
Modified Linear Search for counting '18's in Array:
Count: 2
Modified Linear Search for counting '18's in Linked List:
Count: 2

Process finished with exit code 0
```

T.I.P. TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY

Problem 3. Suppose you have the following sorted list [3, 5, 6, 8, 11, 12, 14, 15, 17, 18] and are using the binary search algorithm. If you wanted to find the key 8, draw a diagram that shows how the searching works per iteration of the algorithm. Prove that your drawing is correct by implementing the algorithm and showing a screenshot of the code and the output console.

DIAGRAM



CODE

```
std::cout << "\n-----------PROBLEM 3 & 4-----------\n\n";
std::cout << "Binary Search for for '8' in Array:\n";
binarySearchArr(dataset2, size, dataFind: 8);

std::cout << "Binary Search for '8' in Linked List:\n";
binarySearchLL(head2, dataFind: 8);
```

```
template <typename T>
void binarySearchArr(T data[], int arrSize,T dataFind) {
    int low = 0;
    int high = arrSize - 1;

    while (low <= high) {
        int middle = getMiddleArr(low, high);

        if (data[middle] == dataFind) {
            std::cout << "Search Success\n";
            return;
        }
        else if (data[middle] > dataFind) {
            high = middle - 1;
        }
        else {
            low = middle + 1;
        }
    }
    std::cout << "Search Failed\n";
}
```

```cpp
template <typename T>
void binarySearchLL(Node<T>* head, T dataFind) {
    Node<T>* start = head;
    Node<T>* last = nullptr;

    while (start != last) { //if start == last, it means that they did not find the data
        Node<T>* middle = getMiddleLL(start, last);

        if (middle->data == dataFind) {
            std::cout << "Search Success\n";
            return;
        }
        else if (middle->data > dataFind) {
            last = middle;
        }
        else { //middle->data < dataFind
            start = middle->next;
        }
    }
    std::cout << "Search Failed\n";
}
```

OUTPUT

```
-----------PROBLEM 3 & 4-----------

Binary Search for'8' in Array:
Search Success
Binary Search for '8' in Linked List:
Search Success

Process finished with exit code 0
```

Problem 4. Modify the binary search algorithm so that the algorithm becomes recursive. Using this new recursive binary search, implement a solution to the same problem for problem 3.

CODE

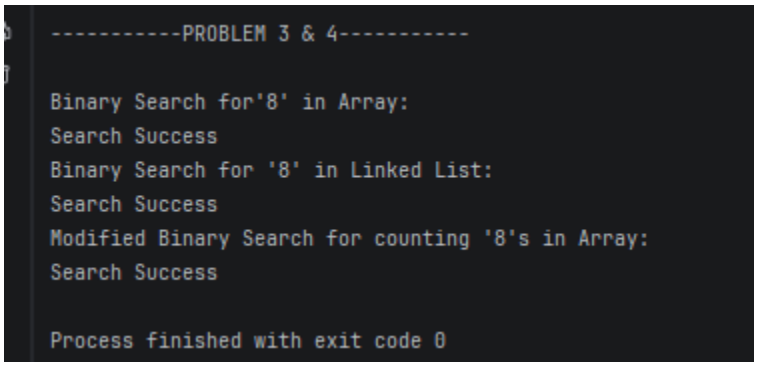
```
    std::cout << "\n-----------PROBLEM 3 & 4-----------\n\n";
    std::cout << "Binary Search for'8' in Array:\n";
    binarySearchArr(dataset2, size, dataFind: 8);
    std::cout << "Binary Search for '8' in Linked List:\n";
    binarySearchLL(head2, dataFind: 8);

    std::cout << "Modified Binary Search for counting '8's in Array:\n";
    binarySearchArrMOD(dataset2, size, dataFind: 8, low: 0, high: size-1);

    return 0;
}
```

```
template <typename T>
void binarySearchArrMOD(T data[], int arrSize,T dataFind, int low, int high) {
    if (low <= high) {
        int middle = getMiddleArr(low, high);
        if (data[middle] == dataFind) {
            std::cout << "Search Success\n";
            return;
        }
        else if (data[middle] > dataFind) {
            high = middle - 1;
            binarySearchArrMOD(data, arrSize, dataFind, low, high);
            return;
        }
        else {
            low = middle + 1;
            binarySearchArrMOD(data, arrSize, dataFind, low, high);
            return;
        }
    }
    std::cout << "Search Failed\n";
}
```

OUTPUT



CONCLUSION

Summary of lessons learned

I've learned a lot from this activity. I understood how the Linear and Binary Search algorithm works and how they differ from each other. Linear Search checks every element one by one until it either finds the data being searched for or returns that it did not find the data. Binary Search is more efficient in searching compared to Linear Search. It'll first get the beginning and last values, and cut the list in half. Then it sees if the middle element is what it's searching for. If the middle element is not it, it will compare if it's greater or less than the data to search for, and depending on this decision it will replace the beginning and last values. This effectively eliminates all unnecessary elements and efficiently isolates the data to search for. If it still didn't find it, it will return that the data to search for is not in the list.

Analysis of the procedure

The procedures follow this sequence; Data Generation, Linear Search for Array and Linked Lists, and Binary Search for Array and Linked Lists. Generating random data every time can be good to save effort in experimenting with your code. Linear Search for Arrays seemingly starts with the first element and just traverses every element until it finds the desired data or doesn't find it. Binary Search Array gets the middle index of the array in reference with the array size. After that, it sees if the middle is the desired data. If not, it'll compare if the middle value is greater or lesser than the desired data. The outcome of this will determine if the starting index or last index gets replaced by the middle index. Then it'll repeat the process until it either finds the data or returns that the data is not in the list. Essentially, Linear Search for Linked Lists is similar to the Linear Search for Array. The significant difference is how we get the middle for Binary Search Linked List. It uses two pointers to point to a node, the first one traverses one node at a time and the second traverses two nodes at a time. Logically, if they make their moves simultaneously, the second pointer will reach the end of the linked list and the first pointer will be at the half-way point. Now this process of getting the middle is repeated and depending on it, it'll replace the starting pointer and the last pointer for the Binary Search.

Analysis of the supplementary activity

The first problem requires us to identify how many comparisons the function will do until it finds the desired data. This is a linear search function for array and linked list, so it means that if the desired data were to be at the very end, it'll do A LOT of comparisons. I put a simple counter that increments every iteration of the while loop. The second problem asks us to modify the linear search to count how many times the element appears in the list. What I did was add a counter for the functions, and instead of it stopping when it finds the desired data it'll only stop until it has fully traversed the list. The third problem is more on showing that you truly understand how the algorithms work, specifically the Binary Search. It's like visualizing what the code really does per iteration, it helps you understand what it really does. For the last problem it asks us to modify the binary search function to be recursive, meaning it calls itself every decision until it either finds the desired data or returns that it hasn't. I solved this by removing the while loop and replacing it as a conditional statement. The most important part is adding the "low" and "high" values as parameters.

Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?

I think I did fair for this activity. What I needed to improve upon is how quick I understand certain concepts because I'm a bit slow on the uptake.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.